Then make it read only

1 message

Pranav Madhu <pranavmadhu8055@gmail.com> Fri, 23 Oct 2026 at 10:31 am
Draft

Here is the updated code. I have added a `readOnly` state to the `IrregularityRow` model. When "Verified & Ok" or "Not Applicable" is auto-selected, this flag is set to `true`, which then disables the `_GridDropdown` for that specific row and grays out its background so the user cannot change the auto-populated value.

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:convert';
import 'package:flutter/services.dart';
import 'package:http/http.dart' as http;
import 'package:intl/intl.dart';

class IrregularityClosureandVerification extends StatefulWidget {
  const IrregularityClosureandVerification({
    super.key,
    this.width,
    this.height,
    this.baseurl,
    this.branchId,
    this.empCode,
  });
  final double? width;
  final double? height;
  final String? baseurl;
  final String? branchId;
  final String? empCode;
  @override
  State<IrregularityClosureandVerification> createState() =>
      _IrregularityClosureandVerificationState();
}

class _IrregularityClosureandVerificationState
    extends State<IrregularityClosureandVerification> {
  final DateFormat _dateFormat = DateFormat('dd-MM-yyyy');
  List<_PledgeOption> _pledges = [];
  _PledgeOption? _selectedPledge;
  _PledgeDetails? _pledgeDetails;
  List<_Transaction> _transactions = [];
  bool _isLoadingPledges = false;
  bool _isLoadingDetails = false;
  bool _isSubmitting = false;
  String? _errorMessage;
  int selectedStatus = 1; // Default to "Verified & Closed"
  final _remarksController = TextEditingController();

  String get _baseUrl => (widget.baseurl ?? '').replaceAll(RegExp(r'/$'), '');
  // ---- Table State Data ---
  final List<IrregularityRow> _irregularityRows = [
    IrregularityRow(1, 'DPN'),
    IrregularityRow(2, 'PAWN TICKET'),
    IrregularityRow(3, 'DUE DILIGENCE'),
    IrregularityRow(4, 'SCHEME SHEET'),
    IrregularityRow(5, 'UNDERTAKING'),
    IrregularityRow(6, 'INVENTORY RELEASE SLIP'),
    IrregularityRow(7, 'PURITY CHECK'),
  ];
  final List<Map<String, dynamic>> _auditorOptions = [
    {'id': 1, 'name': 'Verified & Ok'},
    {'id': 2, 'name': 'Irregularities noticed'},
    {'id': 3, 'name': 'Not Applicable'},
  ];
  @override
  void dispose() {
    _remarksController.dispose();
```

```
    super.dispose();
  }

  @override
  void initState() {
    super.initState();
    _loadPledges();
  }

  Future<void> _loadPledges() async {
    final branchId = widget.branchId?.trim();
    if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) {
      setState(() => _errorMessage = 'Branch and API URL are required.');
      return;
    }
    setState(() {
      _isLoadingPledges = true;
      _errorMessage = null;
    });
    try {
      final response = await http.get(Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'IrrCodeDropdown/$branchId/1',
      ));
      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Unable to load pledge numbers.');
      }
      final result = _resultList(response.body);
      if (!mounted) return;
      setState(() {
        _pledges = result
            .whereType<Map<String, dynamic>>()
            .map(_PledgeOption.fromJson)
            .where((pledge) => pledge.pledgeNo.isNotEmpty)
            .toList();
        _isLoadingPledges = false;
      });
    } catch (_) {
      if (!mounted) return;
      setState(() {
        _isLoadingPledges = false;
        _errorMessage = 'Unable to load pledge numbers.';
      });
    }
  }

  Future<void> _loadPledgeData(_PledgeOption pledge) async {
    final branchId = widget.branchId?.trim();
    if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) return;
    setState(() {
      _selectedPledge = pledge;
      _pledgeDetails = null;
      _transactions = [];
      _isLoadingDetails = true;
      _errorMessage = null;
    });
    try {
      final detailsUri = Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'IrrCodeDropdownDetails/$branchId*${pledge.irrCode}/1',
      );
      final transactionsUri = Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'cus_tranDetails/${pledge.pledgeNo}/1',
      );
      final settlementTypeUri = Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'settlementTypes/${pledge.pledgeNo}/1',
      );
      final getLossUri = Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'lossAmountDetails/${pledge.pledgeNo}*${pledge.irrCode}/1',
      );
      final responses = await Future.wait([
        http.get(detailsUri),
        http.get(transactionsUri),
        http.get(settlementTypeUri),
        http.get(getLossUri),
      ]);
      for (final response in responses) {
        if (response.statusCode < 200 || response.statusCode >= 300) {
          throw Exception('Unable to load pledge data.');
        }
      }
```

```
    final detailRows = _resultList(responses[0].body);
    final transactionRows = _resultList(responses[1].body);
    final settlementTypeRows = _resultList(responses[2].body);
    final lossRows = _resultList(responses[3].body);
    double? lossValue;
    if (lossRows.isNotEmpty && lossRows.first is Map<String, dynamic>) {
      lossValue = (lossRows.first as Map<String, dynamic>)['RES'];
    }
    String? settlementTypesRow;
    if (settlementTypeRows.isNotEmpty &&
        settlementTypeRows.first is Map<String, dynamic>) {
      settlementTypesRow =
          (settlementTypeRows.first as Map<String, dynamic>)['RES'];
    }
    if (!mounted) return;
    setState(() {
      if (detailRows.isNotEmpty && detailRows.first is Map<String, dynamic>) {
        final detail =
            _PledgeDetails.fromJson(detailRows.first as Map<String, dynamic>);
        _pledgeDetails = _PledgeDetails(
          pledgeDate: detail.pledgeDate,
          irrcode: detail.irrcode,
          pledgeValue: detail.pledgeValue,
          irregularityDate: detail.irregularityDate,
          irregularityType: detail.irregularityType,
          settlementDate: detail.settlementDate,
          settlementType: settlementTypesRow,
          lossAmount: detail.lossAmount,
          lossWt: lossValue?.toString(),
        );
      } else {
        _pledgeDetails = null;
      }
      _transactions = transactionRows
          .whereType<Map<String, dynamic>>()
          .map(_Transaction.fromJson)
          .toList();
      _isLoadingDetails = false;
    });
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isLoadingDetails = false;
      _errorMessage = 'Unable to load details for the selected pledge.';
    });
  }
}

// Table Select Data API Fetch
Future<void> _updateSelectDataOptions(int index, int classification) async {
  final row = _irregularityRows[index];
  setState(() {
    row.auditorClassification = classification;
    row.selectData = null; // Reset selection on change
    row.selectDataName = '';
    row.isFetchingOptions = true;
    row.isSelectDataReadOnly = false; // Reset read-only status
  });
  String param = '';
  if (classification == 1 || classification == 3) {
    param = '60*0';
  } else if (classification == 2) {
    param = '8*${row.id}';
  }
  if (param.isNotEmpty) {
    try {
      final response = await http.get(Uri.parse(
          '$_baseUrl/AuditInspectionApi/api/GetInspectionData/PROC_VERIFY_AND_CLOSURE/$param/1'));
      if (response.statusCode == 200) {
        final decoded = jsonDecode(response.body);
        if (decoded != null && decoded['result'] is List) {
          final List<dynamic> result = decoded['result'];
          final List<Map<String, dynamic>> fetchedOptions = result
              .map((e) {
                dynamic rawId = e['0'];
                int id = (rawId is num)
                    ? rawId.toInt()
                    : (int.tryParse(rawId?.toString() ?? '0') ?? 0);
                return {
                  'id': id,
                  'name': e["'----------SELECT----------'"]?.toString() ?? ''
                };
              })
              .where((e) => e['id'] != 0)
```

```
              .toList(); // Filter out '0' default select option
          if (mounted) {
            setState(() {
              row.selectDataOptions = fetchedOptions;

              // --- AUTO SELECT & READ ONLY LOGIC START ---
              if (classification == 1) {
                try {
                  final autoOption = fetchedOptions.firstWhere((e) =>
                    (e['name'] as String).toLowerCase().contains('verified & ok') ||
                    (e['name'] as String).toLowerCase().contains('verified'));
                  row.selectData = autoOption['id'] as int;
                  row.selectDataName = autoOption['name'] as String;
                  row.isSelectDataReadOnly = true; // Lock dropdown
                } catch (_) {}
              } else if (classification == 3) {
                try {
                  final autoOption = fetchedOptions.firstWhere((e) =>
                    (e['name'] as String).toLowerCase().contains('not applicable') ||
                    (e['name'] as String).toLowerCase().contains('applicable'));
                  row.selectData = autoOption['id'] as int;
                  row.selectDataName = autoOption['name'] as String;
                  row.isSelectDataReadOnly = true; // Lock dropdown
                } catch (_) {}
              }
              // --- AUTO SELECT & READ ONLY LOGIC END ---

            });
          }
        }
      }
    } catch (e) {
      debugPrint('Error fetching dropdown options: $e');
    }
  } else {
    if (mounted) {
      setState(() {
        row.selectDataOptions = [];
      });
    }
  }
  if (mounted) {
    setState(() {
      row.isFetchingOptions = false;
    });
  }
}

List<dynamic> _resultList(String body) {
  final decoded = jsonDecode(body);
  return decoded is Map<String, dynamic> && decoded['result'] is List
      ? decoded['result'] as List<dynamic>
      : <dynamic>[];
}

String _formatDate(String? value) {
  if (value == null || value.isEmpty) return '-';
  try {
    return _dateFormat.format(DateTime.parse(value).toLocal());
  } catch (_) {
    return value;
  }
}

String _formatAmount(num? value) => value == null ? '-' : value.toString();

Future<void> _showAlert(String message) async {
  if (!mounted) return;
  await showDialog<void>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Alert'),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('OK'),
        ),
      ],
    ),
  );
}

Future<void> _submitClosure(String flag) async {
```

```
    final pledge = _selectedPledge;
    final branchId = widget.branchId?.trim();
    final empCode = widget.empCode?.trim();
    final remarks = _remarksController.text.trim();


    if (pledge == null || branchId == null || branchId.isEmpty) {
      await _showAlert('Please select a pledge first.');
      return;
    }

    // Check if table auditor classifications are selected (if applicable)
    bool tableIncomplete =
        _irregularityRows.any((row) => row.auditorClassification == null);
    if (tableIncomplete) {
      await _showAlert(
          'Please select AUDITOR CLASSIFICATION for all categories.');
      return;
    }
    if (remarks.isEmpty) {
      await _showAlert('Please enter remarks.');
      return;
    }
    if (empCode == null || empCode.isEmpty) {
      setState(() => _errorMessage = 'Employee code is required.');
      return;
    }
    setState(() {
      _isSubmitting = true;
      _errorMessage = null;
    });
    try {
            List<String> rowStrings = [];

      for (var row in _irregularityRows) {

        final id = row.id;

        final category = row.category.replaceAll(RegExp(r'[~!\*@]'), ' ');

        final rowRemarks = row.remarks.replaceAll(RegExp(r'[~!\*@]'), ' ');
        final spotRect = row.spotRectified ? '1' : '0';
        final auditorClassification=row.auditorClassification;
         final issuename=row.selectDataName;

        rowStrings.add('${id}@${auditorClassification}@${category}@${issuename}@${rowRemarks}@${spotRect}');
      }


      final tableDataString = rowStrings.join('!');

      final pInData = '$branchId~${pledge.irrCode}~${pledge.pledgeNo}~$empCode~$selectedStatus~$remarks~$
tableDataString';

      final response = await http.post(
        Uri.parse('$_baseUrl/AuditInspectionApi/api/PostInseptionCtl'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'flag': 'closure_and_verify_post',
          'pageValue': 'approve*$pInData',
          'paraValue': '1',
        }),
      );

      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Unable to submit closure action.');
      }
      if (!mounted) return;
      final decodedResponse = jsonDecode(response.body);
      final responseMessage = decodedResponse is List &&
              decodedResponse.isNotEmpty &&
              decodedResponse.first is Map<String, dynamic>
          ? '${(decodedResponse.first as Map<String, dynamic>)['RES'] ?? ''}'
          : '';
      setState(() {
        _isSubmitting = false;
        _errorMessage = null;
        _remarksController.clear();
        _selectedPledge = null;
        _pledgeDetails = null;
        _transactions.clear();
        for (var row in _irregularityRows) {
          row.auditorClassification = null;
          row.selectData = null;
```

```
        row.remarks = '';
        row.spotRectified = false;
      }
    });
    await showDialog<void>(
      context: context,
      builder: (context) => AlertDialog(
        title: Text(
          responseMessage.isNotEmpty
              ? responseMessage
              : 'Error: No response message received.',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(),
            child: const Text('OK'),
          ),
        ],
      ),
    );
    _loadPledges();
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isSubmitting = false;
      _errorMessage = 'Unable to submit the closure action.';
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Back',
        icon: const Icon(Icons.arrow_back, color: Color(0xFF1A202C)),
        onPressed: () => Navigator.of(context).maybePop(),
      ),
      centerTitle: true,
      title: const Text(
        'Irregularity Closure & Verification',
        style: TextStyle(
          fontWeight: FontWeight.w600,
          fontSize: 20,
          color: Color(0xFF1A202C),
        ),
      ),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
    ),
    body: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(
              horizontal: 48.0,
              vertical: 32.0,
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (_errorMessage != null)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 16.0),
                    child: Text(
                      _errorMessage!,
                      style: TextStyle(
                        color: Colors.red.shade700,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                _buildGeneralInfoSection(),
                if (_isLoadingDetails)
                  const Padding(
                    padding: EdgeInsets.symmetric(vertical: 40),
                    child: Center(
                      child: CircularProgressIndicator(
                        color: Color(0xFF005FB8),
                      ),
```

```
                    ),
                  ),
                  if (_selectedPledge != null && !_isLoadingDetails) ...[
                    const SizedBox(height: 32),
                    _buildTransactionsTableSection(),
                    const SizedBox(height: 32),
                    _buildIrregularitiesTableSection(),
                    const SizedBox(height: 32),
                    _buildClosureActionsSection(),
                    const SizedBox(height: 32),
                  ],
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildClosureActionsSection() {
    return _SectionContainer(
      title: 'Closure Action',
      icon: Icons.fact_check_outlined,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.start,
            children: [
              _FormTextField(
                'Remarks',
                width: 400,
                controller: _remarksController,
                hintText: 'Enter remarks...',
              ),
              const SizedBox(width: 32),
              _FormDropdown<int>(
                label: 'Status',
                width: 300,
                value: selectedStatus,
                items: const [1, 2],
                displayText: (item) =>
                    item == 1 ? 'Verified & Closed' : 'Closed with Deviation',
                onChanged: (value) {
                  if (value != null) {
                    setState(() {
                      selectedStatus = value;
                    });
                  }
                },
              ),
            ],
          ),
          const SizedBox(height: 20),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              ElevatedButton.icon(
                onPressed: _isSubmitting ? null : () => _submitClosure('0'),
                label: const Text('Confirm'),
                style: ElevatedButton.styleFrom(
                  backgroundColor: const Color(0xFF16803C),
                  foregroundColor: Colors.white,
                  padding:
                      const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(6),
                  ),
                ),
              ),
              const SizedBox(width: 16),
              OutlinedButton.icon(
                onPressed: _isSubmitting ? null : () => context.go('/menuPage'),
                icon: const Icon(Icons.exit_to_app),
                label: const Text('Exit'),
                style: OutlinedButton.styleFrom(
                  foregroundColor: const Color(0xFF2D3748),
                  padding:
                      const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
                  side: const BorderSide(color: Color(0xFFCBD5E0)),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(6),
                  ),
```

```
            ),
          ),
        ],
      ),
    ],
  ),
 );
}

Widget _buildGeneralInfoSection() {
  return _SectionContainer(
    title: 'Pledge Selection & Details',
    icon: Icons.assignment_ind_outlined,
    child: Column(
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            _FormDropdown<_PledgeOption>(
              label: 'Select Pledge No',
              width: 220,
              value: _selectedPledge,
              items: _pledges,
              onChanged: _isLoadingPledges
                  ? null
                  : (value) {
                      if (value != null) _loadPledgeData(value);
                    },
              displayText: (item) => item.pledgeNo,
              hintText: _isLoadingPledges ? 'Loading...' : 'Select...',
            ),
            _FormTextField(
              'Irregularity Code',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? (_pledgeDetails!.irrcode ?? '-')
                  : '-',
            ),
            _FormTextField(
              'Pledge Date',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? _formatDate(_pledgeDetails!.pledgeDate)
                  : '-',
            ),
            _FormTextField(
              'Pledge Value',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? _formatAmount(_pledgeDetails!.pledgeValue)
                  : '-',
            ),
          ],
        ),
        const SizedBox(height: 32),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            _FormTextField(
              'Irregularity Reporting Date',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? _formatDate(_pledgeDetails!.irregularityDate)
                  : '-',
            ),
            _FormTextField(
              'Irregularity Type',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? (_pledgeDetails!.irregularityType ?? '-')
                  : '-',
            ),
            _FormTextField(
              'Loss Amt',
              width: 220,
              readOnly: true,
              hintText:
                  _pledgeDetails != null ? _pledgeDetails!.lossAmount : '-',
            ),
```

```
            _FormTextField(
              'Loss Wt',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null ? _pledgeDetails!.lossWt : '-',
            ),
          ],
        ),
        const SizedBox(height: 32),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            _FormTextField(
              'Pledge Settlement Date',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? _formatDate(_pledgeDetails!.settlementDate)
                  : '-',
            ),
            _FormTextField(
              'Type of Settlement',
              width: 220,
              readOnly: true,
              hintText: _pledgeDetails != null
                  ? (_pledgeDetails!.settlementType ?? '-')
                  : '-',
            ),
            const SizedBox(width: 220),
            const SizedBox(width: 220),
          ],
        ),
      ],
    ),
  );
}

Widget _buildTransactionsTableSection() {
  final totalCredit = _transactions
      .where((transaction) => transaction.type == 'C')
      .fold<num>(
          0, (total, transaction) => total + (transaction.amount ?? 0));
  final totalDebit = _transactions
      .where((transaction) => transaction.type == 'D')
      .fold<num>(
          0, (total, transaction) => total + (transaction.amount ?? 0));
  return _SectionContainer(
    title: 'Transaction Details',
    icon: Icons.receipt_long_outlined,
    padding: EdgeInsets.zero,
    child: Padding(
      padding: const EdgeInsets.all(8.0),
      child: ClipRRect(
        borderRadius: const BorderRadius.only(
          bottomLeft: Radius.circular(12),
          bottomRight: Radius.circular(12),
        ),
        child: SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          child: DataTable(
            decoration: BoxDecoration(
              border: Border.all(color: const Color(0xFFEDF2F7)),
            ),
            headingRowColor:
                MaterialStateProperty.all(const Color(0xFFF9FAFB)),
            dataRowColor: MaterialStateProperty.all(Colors.white),
            dividerThickness: 1,
            horizontalMargin: 24,
            columns: const [
              DataColumn(
                  label: Text('Pledge No',
                      style: TextStyle(
                          fontWeight: FontWeight.w600,
                          fontSize: 12,
                          color: Color(0xFF4A5568)))),
              DataColumn(
                  label: Text('Trans Dt',
                      style: TextStyle(
                          fontWeight: FontWeight.w600,
                          fontSize: 12,
                          color: Color(0xFF4A5568)))),
              DataColumn(
                  label: Text('Credit',
                      style: TextStyle(
```

```
                        fontWeight: FontWeight.w600,
                        fontSize: 12,
                        color: Color(0xFF4A5568)))),
              DataColumn(
                  label: Text('Debit',
                      style: TextStyle(
                          fontWeight: FontWeight.w600,
                          fontSize: 12,
                          color: Color(0xFF4A5568)))),
              DataColumn(
                  label: Text('Description',
                      style: TextStyle(
                          fontWeight: FontWeight.w600,
                          fontSize: 12,
                          color: Color(0xFF4A5568)))),
            ],
            rows: [
              ..._transactions.map((transaction) => DataRow(cells: [
                    DataCell(Text(transaction.pledgeNo,
                        style: const TextStyle(
                            fontSize: 12, color: Color(0xFF2D3748)))),
                    DataCell(Text(_formatDate(transaction.transactionDate),
                        style: const TextStyle(
                            fontSize: 12, color: Color(0xFF2D3748)))),
                    DataCell(Text(
                        transaction.type == 'C'
                            ? _formatAmount(transaction.amount)
                            : '-',
                        style: TextStyle(
                            fontSize: 12,
                            color: transaction.type == 'C'
                                ? Colors.green.shade700
                                : const Color(0xFF2D3748),
                            fontWeight: transaction.type == 'C'
                                ? FontWeight.w600
                                : FontWeight.normal))),
                    DataCell(Text(
                        transaction.type == 'D'
                            ? _formatAmount(transaction.amount)
                            : '-',
                        style: TextStyle(
                            fontSize: 12,
                            color: transaction.type == 'D'
                                ? Colors.red.shade700
                                : const Color(0xFF2D3748),
                            fontWeight: transaction.type == 'D'
                                ? FontWeight.w600
                                : FontWeight.normal))),
                    DataCell(Text(transaction.description,
                        style: const TextStyle(
                            fontSize: 12, color: Color(0xFF2D3748)))),
                  ])),
              DataRow(
                color: MaterialStateProperty.all(const Color(0xFFF1F5F9)),
                cells: [
                  const DataCell(Text('Total',
                      style: TextStyle(
                          fontSize: 12, fontWeight: FontWeight.w700))),
                  const DataCell(SizedBox.shrink()),
                  DataCell(Text(_formatAmount(totalCredit),
                      style: TextStyle(
                          fontSize: 12,
                          color: Colors.green.shade700,
                          fontWeight: FontWeight.w700))),
                  DataCell(Text(_formatAmount(totalDebit),
                      style: TextStyle(
                          fontSize: 12,
                          color: Colors.red.shade700,
                          fontWeight: FontWeight.w700))),
                  const DataCell(SizedBox.shrink()),
                ],
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

// --- Irregularities Assessment Table --
Widget _buildIrregularitiesTableSection() {
  return _SectionContainer(
    title: 'Irregularities Assessment',
```

```
icon: Icons.rule,
padding: EdgeInsets.zero,
child: ClipRRect(
  borderRadius: const BorderRadius.only(
    bottomLeft: Radius.circular(12),
    bottomRight: Radius.circular(12),
  ),
  child: Container(
    color: Colors.white,
    child: Table(
      defaultVerticalAlignment: TableCellVerticalAlignment.middle,
      border: TableBorder.all(color: const Color(0xFFEDF2F7), width: 1.5),
      columnWidths: const {
        0: FlexColumnWidth(1.2),
        1: FlexColumnWidth(1.5),
        2: FlexColumnWidth(3.0),
        3: FlexColumnWidth(1.5),
        4: FlexColumnWidth(0.8),
      },
      children: [
        TableRow(
          decoration: const BoxDecoration(
              color: Color(
                  0xFF1B3245)), // Dark header color matching the image
          children: [
            _buildHeaderCell('IRREGULARITY CATEGORY'),
            _buildHeaderCell('AUDITOR CLASSIFICATION'),
            _buildHeaderCell('SELECT DATA'),
            _buildHeaderCell('REMARKS'),
            _buildHeaderCell('SPOT RECTIFIED'),
          ],
        ),
        ..._irregularityRows.asMap().entries.map((entry) {
          final idx = entry.key;
          final row = entry.value;
          return TableRow(
            children: [
              Padding(
                padding: const EdgeInsets.symmetric(
                    vertical: 12, horizontal: 16),
                child: Text(
                  row.category,
                  style: const TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 11,
                      color: Color(0xFF334155)),
                  textAlign: TextAlign.center,
                ),
              ),
              Padding(
                padding: const EdgeInsets.symmetric(
                    horizontal: 12, vertical: 8),
                child: _GridDropdown<int>(
                  value: row.auditorClassification,
                  items:
                      _auditorOptions.map((e) => e['id'] as int).toList(),
                  displayText: (val) => _auditorOptions.firstWhere(
                      (e) => e['id'] == val)['name'] as String,
                  onChanged: (val) {
                    if (val != null) _updateSelectDataOptions(idx, val);
                  },
                ),
              ),
              Padding(
                padding: const EdgeInsets.symmetric(
                    horizontal: 12, vertical: 8),
                child: row.isFetchingOptions
                    ? const Center(
                        child: SizedBox(
                          width: 16,
                          height: 16,
                          child: CircularProgressIndicator(
                              strokeWidth: 2, color: Color(0xFF005FB8)),
                        ),
                      )
                    : _GridDropdown<int>(
                        value: row.selectData,
                        readOnly: row.isSelectDataReadOnly, // Lock when set to readOnly
                        items: row.selectDataOptions
                            .map((e) => e['id'] as int)
                            .toList(),
                        displayText: (val) => row.selectDataOptions
                                .firstWhere((e) => e['id'] == val)['name']
                            as String,
```

```
                                onChanged: row.isSelectDataReadOnly
                                  ? null
                                  : (val) {
                                    setState(() {
                                      row.selectData = val;
                                      row.selectDataName=row.selectDataOptions.firstWhere((e) => e['id'] == val)
['name'];
                                    });
                                  },
                              ),
                        ),
                        Padding(
                          padding: const EdgeInsets.symmetric(
                              horizontal: 12, vertical: 8),
                          child: SizedBox(
                            height: 36,
                            child: TextFormField(
                              initialValue: row.remarks,
                              onChanged: (val) => row.remarks = val,
                              style: const TextStyle(fontSize: 11),
                              decoration: InputDecoration(
                                hintText: 'Enter Remarks',
                                hintStyle: const TextStyle(
                                    fontSize: 11, color: Colors.grey),
                                contentPadding:
                                    const EdgeInsets.symmetric(horizontal: 10),
                                border: OutlineInputBorder(
                                    borderRadius: BorderRadius.circular(4),
                                    borderSide:
                                        const BorderSide(color: Color(0xFFCBD5E0))),
                                enabledBorder: OutlineInputBorder(
                                    borderRadius: BorderRadius.circular(4),
                                    borderSide:
                                        const BorderSide(color: Color(0xFFCBD5E0))),
                                focusedBorder: OutlineInputBorder(
                                    borderRadius: BorderRadius.circular(4),
                                    borderSide:
                                        const BorderSide(color: Color(0xFF005FB8))),
                              ),
                            ),
                          ),
                        ),
                        Padding(
                          padding: const EdgeInsets.symmetric(
                              horizontal: 4, vertical: 8),
                          child: Row(
                            mainAxisAlignment: MainAxisAlignment.center,
                            children: [
                              SizedBox(
                                width: 24,
                                height: 24,
                                child: Checkbox(
                                  value: row.spotRectified,
                                  shape: RoundedRectangleBorder(
                                      borderRadius: BorderRadius.circular(3)),
                                  side: const BorderSide(
                                      color: Color(0xFFCBD5E0), width: 1.5),
                                  onChanged: (v) {
                                    setState(() {
                                      row.spotRectified = v ?? false;
                                    });
                                  },
                                ),
                              ),
                              const SizedBox(width: 4),
                              const Text('Spot rectified',
                                  style: TextStyle(
                                      fontSize: 10, color: Color(0xFF4A5568))),
                            ],
                          ),
                        ),
                      ],
                    );
                  }),
                ],
              ),
            ),
          ),
        );
  }

  Widget _buildHeaderCell(String text) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 14.0, horizontal: 8.0),
```

```
      child: Text(
        text,
        textAlign: TextAlign.center,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 10,
          fontWeight: FontWeight.w600,
          letterSpacing: 0.5,
        ),
      ),
    );
  }
}

// --- Reusable UI Components based on Reference Design
class _SectionContainer extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final EdgeInsetsGeometry padding;
  const _SectionContainer({
    required this.title,
    required this.icon,
    required this.child,
    this.padding = const EdgeInsets.all(24.0),
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 20,
            offset: const Offset(0, 5),
          ),
        ],
        border: Border.all(color: const Color(0xFFEDF2F7)),
      ),
      child: Column(
        children: [
          const SizedBox(height: 8),
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 16.0,
            ),
            decoration: const BoxDecoration(
              color: Color(0xFFF9FAFB),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(12),
                topRight: Radius.circular(12),
              ),
              border: Border(
                bottom: BorderSide(color: Color(0xFFEDF2F7)),
              ),
            ),
            child: Row(
              children: [
                Icon(
                  icon,
                  size: 22,
                  color: const Color(0xFF4A5568),
                ),
                const SizedBox(width: 12),
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFF2D3748),
                  ),
                ),
              ],
            ),
          ),
          Padding(
            padding: padding,
            child: child,
          ),
        ],
      ),
```

```
    );
  }
}

class _GridDropdown<T> extends StatelessWidget {
  final T? value;
  final List<T> items;
  final void Function(T?)? onChanged;
  final String Function(T) displayText;
  final String hintText;
  final bool readOnly; // Added flag for read-only styling

  const _GridDropdown({
    required this.value,
    required this.items,
    required this.onChanged,
    required this.displayText,
    this.hintText = 'Select...',
    this.readOnly = false,
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      height: 36,
      padding: const EdgeInsets.symmetric(horizontal: 10),
      decoration: BoxDecoration(
        color: readOnly ? const Color(0xFFF7F8FA) : Colors.white, // Gray out if readOnly
        borderRadius: BorderRadius.circular(4),
        border: Border.all(color: const Color(0xFFCBD5E0)),
      ),
      child: DropdownButtonHideUnderline(
        child: DropdownButton<T>(
          isExpanded: true,
          value: items.contains(value) ? value : null,
          hint: Text(hintText,
              style: const TextStyle(fontSize: 11, color: Colors.grey)),
          icon: Icon(Icons.keyboard_arrow_down,
              size: 18, color: readOnly ? Colors.grey : const Color(0xFF4A5568)),
          items: items.map((e) {
            return DropdownMenuItem(
              value: e,
              child: Text(
                displayText(e),
                style: TextStyle(
                  fontSize: 11,
                  color: readOnly ? Colors.grey.shade700 : const Color(0xFF1A202C)
                ),
                overflow: TextOverflow.ellipsis,
              ),
            );
          }).toList(),
          onChanged: readOnly ? null : onChanged, // Disable selection callback if readOnly
        ),
      ),
    );
  }
}

class _FormDropdown<T> extends StatelessWidget {
  final String label;
  final double width;
  final T? value;
  final List<T> items;
  final void Function(T?)? onChanged;
  final String Function(T) displayText;
  final String hintText;
  const _FormDropdown({
    Key? key,
    required this.label,
    required this.width,
    required this.value,
    required this.items,
    required this.onChanged,
    required this.displayText,
    this.hintText = 'Select...',
  }) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
```

```
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
          ),
          const SizedBox(height: 8),
          Container(
            height: 36,
            padding: const EdgeInsets.symmetric(horizontal: 10),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(6),
              border: Border.all(color: const Color(0xFFCBD5E0)),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.02),
                  blurRadius: 4,
                  offset: const Offset(0, 2),
                ),
              ],
            ),
            child: DropdownButtonHideUnderline(
              child: DropdownButton<T>(
                hint: Text(
                  hintText,
                  style: const TextStyle(fontSize: 12, color: Colors.grey),
                ),
                value: value,
                isExpanded: true,
                icon:
                    const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
                items: items.map((item) {
                  return DropdownMenuItem<T>(
                    value: item,
                    child: Text(
                      displayText(item),
                      style: const TextStyle(fontSize: 12),
                      overflow: TextOverflow.ellipsis,
                    ),
                  );
                }).toList(),
                onChanged: onChanged,
                style: const TextStyle(
                  fontSize: 12,
                  color: Color(0xFF1A202C),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _FormTextField extends StatelessWidget {
  final String label;
  final double width;
  final bool readOnly;
  final String? hintText;
  final TextEditingController? controller;
  const _FormTextField(
    this.label, {
    super.key,
    required this.width,
    this.readOnly = false,
    this.hintText,
    this.controller,
  });
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
```

```
            ),
          ),
          const SizedBox(height: 8),
          Container(
              height: 36,
              decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(6),
                boxShadow: [
                  BoxShadow(
                    color: Colors.black.withOpacity(0.02),
                    blurRadius: 4,
                    offset: const Offset(0, 2),
                  ),
                ],
              ),
              child: TextFormField(
                controller: controller,
                readOnly: readOnly,
                style: const TextStyle(
                  fontSize: 12,
                  color: Colors.black,
                  fontWeight: FontWeight.w500,
                ),
                decoration: InputDecoration(
                  filled: true,
                  fillColor: readOnly ? const Color(0xFFF7F8FA) : Colors.white,
                  contentPadding: const EdgeInsets.symmetric(horizontal: 10),
                  hintText: (hintText?.isNotEmpty ?? false) ? hintText : '-',
                  hintStyle: TextStyle(
                    fontSize: 12,
                    color: readOnly ? const Color(0xFF2D3748) : Colors.grey,
                  ),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
                  ),
                  enabledBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
                  ),
                  focusedBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(
                      color: Color(0xFF005FB8),
                      width: 1.5,
                    ),
                  ),
                ),
              )),
        ],
      ),
    );
  }
}

// --- Data Models
class IrregularityRow {
  final int id;
  final String category;
  int? auditorClassification;
  int? selectData;
  String remarks = '';
  bool spotRectified = false;
  String selectDataName='';
  List<Map<String, dynamic>> selectDataOptions = [];
  bool isFetchingOptions = false;
  bool isSelectDataReadOnly = false; // Flag to track read-only state for selectData

  IrregularityRow(this.id, this.category);
}

class _PledgeOption {
  const _PledgeOption({required this.irrCode, required this.pledgeNo});
  final String irrCode;
  final String pledgeNo;
  factory _PledgeOption.fromJson(Map<String, dynamic> json) {
    final rawIrrCode = '${json['IRR_CODE'] ?? ''}';
    final actualIrrCode =
        rawIrrCode.contains('~') ? rawIrrCode.split('~')[0] : rawIrrCode;
    return _PledgeOption(
      irrCode: actualIrrCode,
      pledgeNo: '${json['PLEDGE_NO'] ?? ''}',
    );
  }
```

```
  @override
  bool operator ==(Object other) =>
      other is _PledgeOption &&
      other.irrCode == irrCode &&
      other.pledgeNo == pledgeNo;
  @override
  int get hashCode => Object.hash(irrCode, pledgeNo);
}

class _PledgeDetails {
  const _PledgeDetails({
    this.pledgeDate,
    this.pledgeValue,
    this.irrcode,
    this.irregularityDate,
    this.irregularityType,
    this.settlementDate,
    this.settlementType,
    this.lossAmount,
    this.lossWt,
  });
  final String? pledgeDate;
  final num? pledgeValue;
  final String? irregularityDate;
  final String? irregularityType;
  final String? settlementDate;
  final String? settlementType;
  final String? lossAmount;
  final String? irrcode;
  final String? lossWt;
  factory _PledgeDetails.fromJson(Map<String, dynamic> json) => _PledgeDetails(
        pledgeDate: json['TRA_DT']?.toString(),
        irrcode: json['IRR_CODE']?.toString(),
        pledgeValue: _asNum(json['PLEDGE_VAL']),
        irregularityDate: json['IRR_RPT_DATE']?.toString(),
        irregularityType: json['STATUS']?.toString(),
        settlementDate: json['CLOSE_DT']?.toString(),
        lossAmount: json['LOSS'].toString(),
      );
}

class _Transaction {
  const _Transaction({
    required this.pledgeNo,
    required this.transactionDate,
    required this.amount,
    required this.type,
    required this.description,
  });
  final String pledgeNo;
  final String? transactionDate;
  final num? amount;
  final String type;
  final String description;
  factory _Transaction.fromJson(Map<String, dynamic> json) => _Transaction(
        pledgeNo: '${json['PLEDGE_NO'] ?? ''}',
        transactionDate: json['TRA_DT']?.toString(),
        amount: _asNum(json['AMOUNT']),
        type: '${json['TYPE'] ?? ''}'.toUpperCase(),
        description: '${json['DESCR'] ?? ''}',
      );
}

num? _asNum(dynamic value) => value is num ? value : num.tryParse('$value');
```